

1. MySQL的执行原理

1.1.单表访问之索引合并

我们前边说过MySQL在一般情况下执行一个查询时最多只会用到单个二级索引，但存在有特殊情况，在这些特殊情况下也可能在一个查询中使用到多个二级索引，MySQL中这种使用到多个索引来完成一次查询的执行方法称之为：**索引合并/index merge**，具体的索引合并算法有下边三种。

1.1.1.Intersection合并

Intersection翻译过来的意思是交集。这里是说某个查询可以使用多个二级索引，将从多个二级索引中查询到的结果取交集，比方说下边这个查询：

```
SELECT * FROM order_exp WHERE order_no = 'a' AND expire_time = 'b';
```

对象 order_exp @demo_explain (...)						
新建	保存	另存为	添加栏位	插入栏位	删除栏位	主键
上移	下移					
栏位	索引	外键	触发器	选项	注释	SQL 预览
名	类型	长度	小数点	不是 null		
id	bigint	22	0	☒	1	
order_no	varchar	50	0	☒		
order_note	varchar	100	0	☒		
insert_time	datetime	0	0	☒		
expire_duration	bigint	22	0	☒		
expire_time	datetime	0	0	☒		
order_status	smallint	6	0	☒		

对象 order_exp @demo_explain (...)			
新建	保存	另存为	添加索引
删除索引			
栏位	索引	外键	触发器
选项	注释	SQL 预览	
名	栏位	索引类型	索引方法
u_idx_day_status	insert_time, order_status, expire_time	Unique	BTREE
idx_order_no	order_no	Normal	BTREE
idx_expire_time	expire_time	Normal	BTREE

假设这个查询使用Intersection合并的方式执行的话，那这个过程就是这样的：

- 从idx_order_no二级索引对应的B+树中取出order_no='a'的相关记录。
- 从idx_expire_time二级索引对应的B+树中取出expire_time='b'的相关记录。

二级索引的记录都是由索引列 + 主键构成的，所以我们可以计算出这两个结果集中id值的交集。

按照上一步生成的id值列表进行回表操作，也就是从聚簇索引中把指定id值的完整用户记录取出来，返回给用户。

为啥不直接使用idx_order_no或者idx_expire_time只根据某个搜索条件去读取一个二级索引，然后回表后再过滤另外一个搜索条件呢？这里要分析一下两种查询执行方式之间需要的成本代价。

只读取一个二级索引的成本：

1.按照某个搜索条件读取一个二级索引

2.根据从该二级索引得到的主键值进行回表操作

3.然后再过滤其他的搜索条件

读取多个二级索引之后取交集成本：

1.按照不同的搜索条件分别读取不同的二级索引

2.将从多个二级索引得到的主键值取交集

3.最后根据主键值进行回表操作。

虽然读取多个二级索引比读取一个二级索引消耗性能，但是大部分情况下读取二级索引的操作是顺序I/O，而回表操作是随机I/O，所以如果只读取一个二级索引时需要回表的记录数特别多，而读取多个二级索引之后取交集的记录数非常少，当节省的因为回表而造成的性能损耗比访问多个二级索引带来的性能损耗更高时，读取多个二级索引后取交集比只读取一个二级索引的成本更低。

所以MySQL在某些特定的情况下才可能会使用到Intersection索引合并，哪些情况呢？

1.1.1.1.等值匹配

二级索引列必须是等值匹配的情况

对于联合索引来说，在联合索引中的每个列都必须等值匹配，不能出现只匹配部分列的情况。

而下边这两个查询就不能进行Intersection索引合并：

```
SELECT * FROM order_exp WHERE order_no > 'a' AND expire_time = 'a'
```

```
SELECT * FROM order_exp WHERE order_no = 'a' AND insert_time = 'a';
```

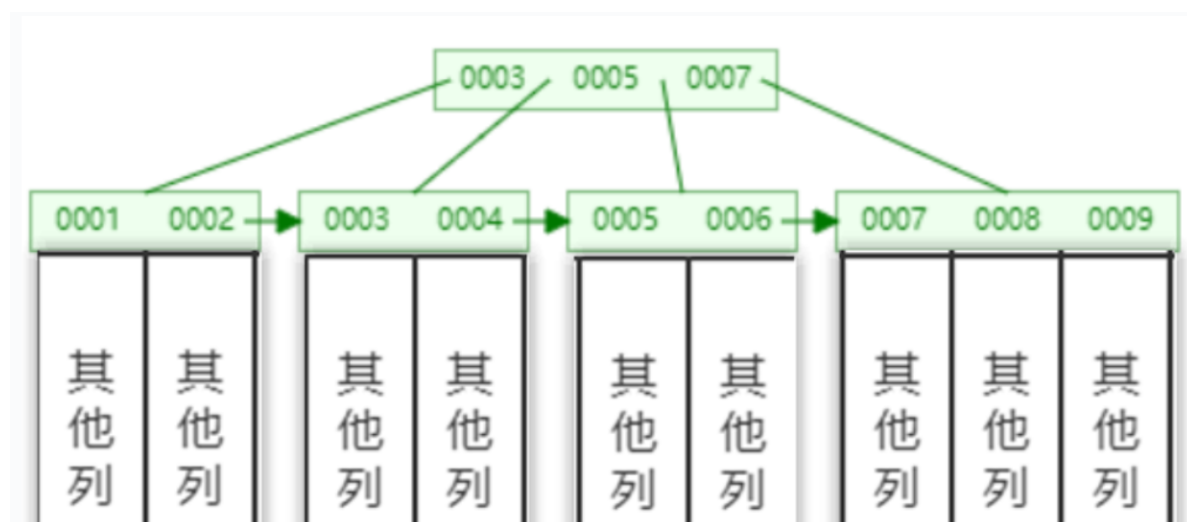
第一个查询是因为对order_no进行了范围匹配

第二个查询是因为insert_time使用到的联合索引u_idx_day_status中的order_status和expire_time列并没有出现在搜索条件中，所以这两个查询不能进行Intersection索引合并。

1.1.1.2.主键列可以是范围匹配

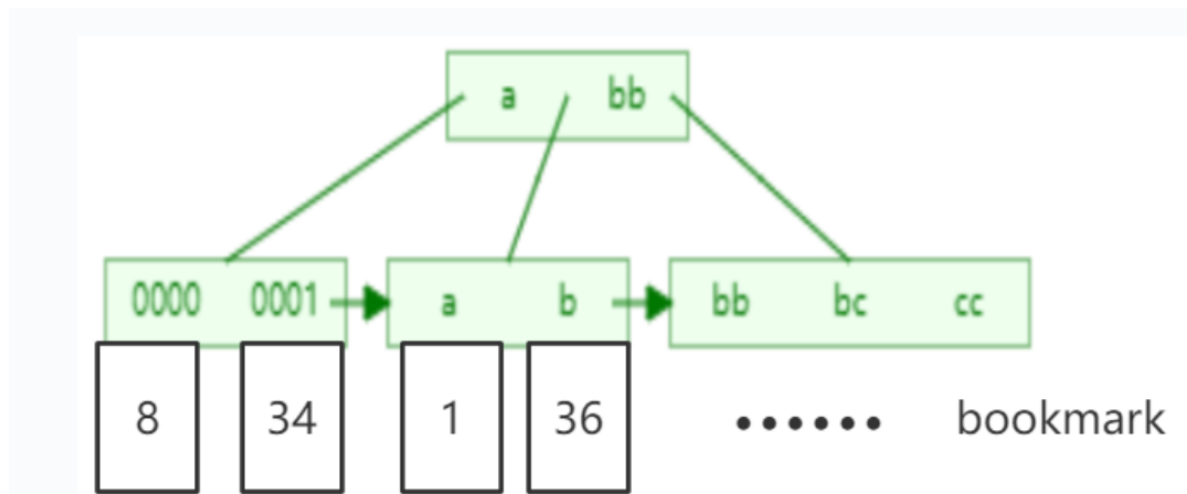
比方说下边这个查询可能用到主键和u_idx_day_status进行Intersection索引合并的操作：

```
SELECT * FROM order_exp WHERE id > 100 AND expire_time = 'a';
```



因为主键的索引是有序的，按照有序的主键值去回表取记录有个专有名词，叫：Rowid Ordered Retrieval，简称 **ROR**。

而二级索引的用户记录是由索引列 + 主键构成的，所以根据范围匹配出来的主键就是乱序的，导致回表开销很大。



那为什么在二级索引列都是等值匹配的情况下也可能使用Intersection索引合并，是因为只有在这种情况下根据二级索引查询出的结果集是按照主键值排序的。

Intersection索引合并会把从多个二级索引中查询出的主键值求交集，如果从各个二级索引中查询的到的结果集本身就是已经按照主键排好序的，那么求交集的过程就很容易。

当然，上边说的两种情况只是发生Intersection索引合并的必要条件，不是充分条件。也就是说即使符合Intersection的条件，也不一定发生Intersection索引合并，这得看优化器的心情（判断）。

优化器只有在单独根据搜索条件从某个二级索引中获取的记录数太多，导致回表开销太大，而通过Intersection索引合并后需要回表的记录数大大减少时才会使用Intersection索引合并。

1.1.2.Union合并

我们在写查询语句时经常想把既符合某个搜索条件的记录取出来，也把符合另外的某个搜索条件的记录取出来，我们说这些不同的搜索条件之间是OR关系。有时候OR关系的不同搜索条件会使用到不同的索引，比方说这样：

```
SELECT * FROM order_exp WHERE order_no = 'a' OR expire_time = 'b'
```

Intersection是交集的意思，这适用于使用不同索引的搜索条件之间使用AND连接起来的情况；Union是并集的意思，适用于使用不同索引的搜索条件之间使用OR连接起来的情况。与Intersection索引合并类似，MySQL在某些特定的情况下才可能会使用到Union索引合并：

1.1.2.1.等值匹配

分析同Intersection合并

1.1.2.2.主键列可以是范围匹配

分析同Intersection合并

1.1.1.3.使用Intersection索引合并的搜索条件

就是搜索条件的某些部分使用Intersection索引合并的方式得到的主键集合和其他方式得到的主键集合取交集，比方说这个查询：

```
SELECT * FROM order_exp WHERE insert_time = 'a' AND order_status = 'b' AND
expire_time = 'c'
OR (order_no = 'a' AND expire_time = 'b');
```

优化器可能采用这样的方式来执行这个查询：

- 1、先按照搜索条件order_no = 'a' AND expire_time = 'b'从索引idx_order_no和idx_expire_time中使用Intersection索引合并的方式得到一个主键集合。
- 2、再按照搜索条件 insert_time = 'a' AND order_status = 'b' AND expire_time = 'c'从联合索引u_idx_day_status中得到另一个主键集合。
- 3、采用Union索引合并的方式把上述两个主键集合取并集，然后进行回表操作，将结果返回给用户。

当然，查询条件符合了这些情况也不一定就会采用Union索引合并，也得看优化器的心情。优化器只有在单独根据搜索条件从某个二级索引中获取的记录数比较少，通过Union索引合并后进行访问的代价比全表扫描更小时才会使用Union索引合并。

1.1.3.Sort-Union合并

Union索引合并的使用条件太苛刻，必须保证各个二级索引列在进行等值匹配的条件下才可能被用到，比方说下边这个查询就无法使用到Union索引合并：

```
SELECT * FROM order_exp WHERE order_no< 'a' OR expire_time> 'z'
```

这是因为根据order_no<'a'从idx_order_no索引中获取的二级索引记录的主键值不是排好序的，

同时根据expire_time>'z'从idx_expire_time索引中获取的二级索引记录的主键值也不是排好序的，但是order_no<'a'和expire_time>'z'这两个条件又特别让我们动心，所以我们可以这样：

- 1、先根据order_no<'a'条件从idx_order_no二级索引中获取记录，并按照记录的主键值进行排序
- 2、再根据expire_time>'z'条件从idx_expire_time二级索引中获取记录，并按照记录的主键值进行排序
- 3、因为上述的两个二级索引主键值都是排好序的，剩下的操作和Union索引合并方式就一样了。

上述这种先按照二级索引记录的主键值进行排序，之后按照Union索引合并方式执行的方式称之为Sort-Union索引合并，很显然，这种Sort-Union索引合并比单纯的Union索引合并多了一步对二级索引记录的主键值排序的过程。

当然，查询条件符合了这些情况也不一定就会采用Sort-Union索引合并，也得看优化器的心情。优化器只有在单独根据搜索条件从某个二级索引中获取的记录数比较少，通过Sort-Union索引合并后进行访问的代价比全表扫描更小时才会使用Sort-Union索引合并。

1.1.4.联合索引替代Intersection索引合并

```
SELECT * FROM order_exp WHERE order_no= 'a' And expire_time= 'z';
```

这个查询之所以可能使用Intersection索引合并的方式执行，还不是因为idx_order_no和idx_expire_time是两个单独的B+树索引，要是把这两个列搞一个联合索引，那直接使用这个联合索引就把事情搞定了，何必用啥索引合并呢，就像这样：

```
ALTER TABLE order_exp drop index idx_order_no;
ALTER TABLE order_exp drop idx_expire_time;
ALTER TABLE add index idx_order_no_expire_time(order_no,expire_time);
```

名	栏位	索引类型	索引方法
u_idx_day_status1	insert_time, order_status, expire_time	Unique	BTREE
idx_order_no_expire_time	order_no, expire_time	Normal	BTREE

这样我们把idx_order_no,idx_expire_time都干掉，再添加一个联合索引idx_order_no_expire_time，使用这个联合索引进行查询简直是又快又好，既不用多读一棵B+树，也不用合并结果。

1.2.连接查询

搞数据库一个避不开的概念就是Join，翻译成中文就是连接。使用的时候常常陷入下边两种误区：

误区一：业务至上，管他三七二十一，再复杂的查询也用在在一个连接语句中搞定。

误区二：敬而远之，上次慢查询就是因为使用了连接导致的，以后再也不敢用了。

所以我们来学习一下连接的原理，才能在工作中用好SQL连接。

1.2.1.连接简介

1.2.1.1.连接的本质

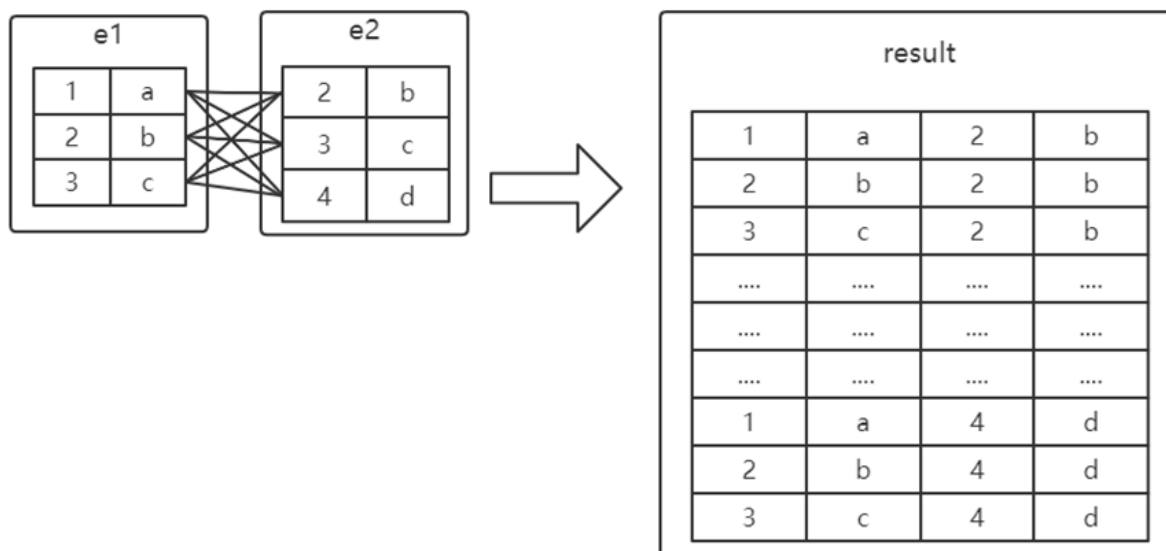
为了方便讲述，我们建立两个简单的演示表并给它们写入数据：

```
CREATE TABLE e1 (m1 int, n1 char(1));
CREATE TABLE e2 (m2 int, n2 char(1));
INSERT INTO e1 VALUES(1, 'a'), (2, 'b'), (3, 'c');
INSERT INTO e2 VALUES(2, 'b'), (3, 'c'), (4, 'd');
```

m1	n1	m2	n2
1	a	2	b
2	b	3	c
3	c	4	d

连接的本质就是把各个连接表中的记录都取出来依次匹配的组合加入结果集并返回给用户。

所以我们将e1和e2两个表连接起来的过程如下图所示：



这个过程看起来就是把e1表的记录和e2的记录连起来组成新的更大的记录，所以这个查询过程称之为连接查询。连接查询的结果集中包含一个表中的每一条记录与另一个表中的每一条记录相互匹配的组合，像这样的结果集就可以称之为 **笛卡尔积**。

因为表e1中有3条记录，表e2中也有3条记录，所以这两个表连接之后的笛卡尔积就有 $3 \times 3 = 9$ 行记录。

在MySQL中，连接查询的语法很随意，只要在FROM语句后边跟多个表名就好了，比如我们把e1表和e2表连接起来的查询语句可以写成这样：

```
SELECT * FROM e1, e2;
```

1.2.1.2.连接过程简介

我们可以连接任意数量张表，但是如果没有任何限制条件的话，这些表连接起来产生的笛卡尔积可能是非常巨大的。比方说3个100行记录的表连接起来产生的笛卡尔积就有 $100 \times 100 \times 100 = 1000000$ 行数据！所以在连接的时候过滤掉特定记录组合是有必要的，在连接查询中的过滤条件可以分成两种，比方说下边这个查询语句：

```
SELECT * FROM e1, e2 WHERE e1.m1 > 1 AND e1.m1 = e2.m2 AND e2.n2 < 'd';
```

涉及单表的条件

$e1.m1 > 1$ 是只针对e1表的过滤条件

$e2.n2 < 'd'$ 是只针对e2表的过滤条件。

涉及两表的条件

比如类似 $e1.m1 = e2.m2$ ，这些条件中涉及到了两个表。

看一下携带过滤条件的连接查询的大致执行过程在这个查询中我们指明了这三个过滤条件：

- $e1.m1 > 1$
- $e1.m1 = e2.m2$
- $e2.n2 < 'd'$

那么这个连接查询的大致执行过程如下：

确定驱动表(t1)

首先确定第一个需要查询的表，这个表称之为 **驱动表**。单表中执行查询语句只需要选取代价最小的那种访问方法去执行单表查询语句就好了（就是说之前从执行计划中找const、ref、ref_or_null、range、index、all等等这些执行方法中选取代价最小的去执行查询）。

此处假设使用e1作为驱动表，那么就需要到e1表中找满足e1.m1 > 1的记录，因为表中的数据太少，我们也没在表上建立二级索引，所以此处查询e1表的访问方法就设定为all，也就是采用全表扫描的方式执行单表查询。

遍历驱动表结果，到被驱动表(t2)中查找匹配记录

针对上一步骤中从驱动表产生的结果集中的每一条记录，分别需要到e2表中查找匹配的记录，所谓匹配的记录，指的是符合过滤条件的记录。

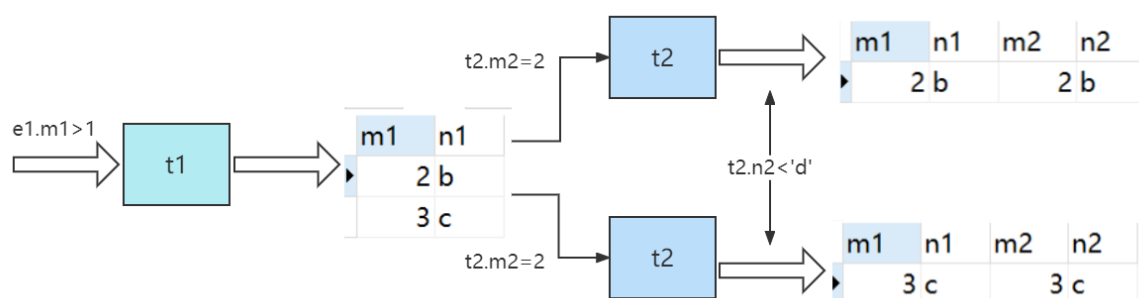
因为是根据e1表中的记录去找e2表中的记录，所以e2表也可以被称之为 **被驱动表**。上一步骤从驱动表中得到了2条记录，所以需要查询2次e2表。

此时涉及两个表的列的过滤条件e1.m1 = e2.m2就派上用场了

当e1.m1 = 2时，过滤条件e1.m1 = e2.m2就相当于e2.m2 = 2，所以此时e2表相当于有了e2.m2 = 2、e2.n2 < 'd'这两个过滤条件，然后到e2表中执行单表查询。

当e1.m1 = 3时，过滤条件e1.m1 = e2.m2就相当于e2.m2 = 3，所以此时e2表相当于有了e2.m2 = 3、e2.n2 < 'd'这两个过滤条件，然后到e2表中执行单表查询。

所以整个连接查询的执行过程就如下图所示：



也就是说整个连接查询最后的结果只有两条符合过滤条件的记录：

从上边两个步骤可以看出来，这个两表连接查询共需要查询1次e1表，2次e2表。

当然这是在特定的过滤条件下的结果，如果我们把e1.m1 > 1这个条件去掉，那么从e1表中查出的记录就有3条，就需要查询3次e2表了。也就是说在两表连接查询中，**驱动表只需要访问一次，被驱动表可能被访问多次**。

1.2.1.3.内连接和外连接

为了大家更好理解后边内容，我们创建两个有现实意义的表，并插入一些数据：

```
CREATE TABLE student (  
    number INT NOT NULL AUTO_INCREMENT COMMENT '学号',  
    name VARCHAR(5) COMMENT '姓名',  
    major VARCHAR(30) COMMENT '专业',  
    PRIMARY KEY (number)  
) Engine=InnoDB CHARSET=utf8 COMMENT '客户信息表';
```

```
CREATE TABLE score (
    number INT COMMENT '学号',
    subject VARCHAR(30) COMMENT '科目',
    score TINYINT COMMENT '成绩',
    PRIMARY KEY (number, subject)
) Engine=InnoDB CHARSET=utf8 COMMENT '客户成绩表';
```

两张表插入以下数据

number	name	major
2022001	ma	计算机科学与技术
2022002	lian	网络工程
2022003	li	软件工程
2022004	yan	计算机科学与技术

number	subject	score
2022001	数据结构和算法	100
2022001	网络通讯原理	98
2022002	网络安全	95
2022002	数据结构和算法	96
2022003	离散数学	88

现在我们把每个学生的考试成绩都查询出来就需要进行两表连接了（因为score中没有姓名信息，所以不能单纯只查询score表）。连接过程就是从student表中取出记录，在score表中查找number相同的成绩记录，所以过滤条件就是student.number = socre.number，整个查询语句就是这样：

```
SELECT s1.number, s1.name, s2.subject, s2.score FROM student AS s1,score AS s2
WHERE s1.number = s2.number;
```

信息	结果1	概况	状态
number	name	subject	score
▶ 2022001	ma	数据结构和算法	100
2022001	ma	网络通讯原理	98
2022002	lian	数据结构和算法	96
2022002	lian	网络安全	95
2022003	li	离散数学	88

从上述查询结果中我们可以看到，各个同学对应的各科成绩就都被查出来了，可是有个问题，yan同学，也就是学号为20200904的同学因为某些原因没有参加考试，所以在score表中没有对应的成绩记录。

如果老师想查看所有同学的考试成绩，即使是缺考的同学也应该展示出来，但是到目前为止我们介绍连接查询是无法完成这样的需求的。我们稍微思考一下这个需求，其本质是想：**驱动表中的记录即使在被驱动表中没有匹配的记录，也仍然需要加入到结果集**。为了解决这个问题，就有了**内连接**和**外连接**的概念：

对于内连接的两个表，驱动表中的记录在被驱动表中找不到匹配的记录，该记录不会加入到最后的结果集，我们上边提到的连接都是所谓的内连接。

对于外连接的两个表，驱动表中的记录即使在被驱动表中没有匹配的记录，也仍然需要加入到结果集。

在MySQL中，根据选取驱动表的不同，外连接仍然可以细分为2种：

左外连接，选取左侧的表为驱动表。

右外连接，选取右侧的表为驱动表。

可是这样仍然存在问题，即使对于外连接来说，有时候我们也并不想把驱动表的全部记录都加入到最后的结果集。

这就犯难了，怎么办？把过滤条件分为两种就可以就解决这个问题了，所以放在不同地方的过滤条件是有不同语义的：

WHERE子句中的过滤条件

WHERE子句中的过滤条件就是我们平时见的那种，不论是内连接还是外连接，凡是不符合WHERE子句中的过滤条件的记录都不会被加入最后的结果集。

ON子句中的过滤条件

对于外连接的驱动表的记录来说，如果无法在被驱动表中找到匹配ON子句中的过滤条件的记录，那么该记录仍然会被加入到结果集中，对应的被驱动表记录的各个字段使用NULL值填充。

需要注意的是，这个ON子句是专门为外连接驱动表中的记录在被驱动表找不到匹配记录时应不应该把该记录加入结果集这个场景下提出的，所以如果把ON子句放到内连接中，MySQL会把它和WHERE子句一样对待，也就是说：内连接中的WHERE子句和ON子句是等价的。

一般情况下，我们都把只涉及单表的过滤条件放到WHERE子句中，把涉及两表的过滤条件都放到ON子句中，我们也一般把放到ON子句中的过滤条件也称之为连接条件。

左（外）连接的语法

左（外）连接的语法还是挺简单的，比如我们要把e1表和e2表进行左外连接查询可以这么写：

```
SELECT * FROM e1 LEFT [OUTER] JOIN e2 ON 连接条件 [WHERE 普通过滤条件];
```

其中中括号里的OUTER单词是可以省略的。

对于LEFTJOIN类型的连接来说：

我们把放在左边的表称之为**外表或者驱动表**

右边的表称之为**内表或者被驱动表**。

所以上述例子中e1就是外表或者驱动表，e2就是内表或者被驱动表。需要注意的是，对于左（外）连接和右（外）连接来说，必须使用ON子句来指出连接条件。了解了左（外）连接的基本语法之后，再次回到我们上边那个现实问题中来，看看怎样写查询语句才能把所有的客户的成绩信息都查询出来，即使是缺考的考生也应该被放到结果集中：

```
SELECT s1.number, s1.name, s2.subject, s2.score FROM student AS s1 LEFT JOIN score AS s2 ON s1.number = s2.number;
```

信息	结果1	概况	状态
number	name	subject	score
▶ 2022001	ma	数据结构和算法	100
2022001	ma	网络通讯原理	98
2022002	lian	数据结构和算法	96
2022002	lian	网络安全	95
2022003	li	离散数学	88
2022004	yan	(Null)	(Null)

从结果集中可以看出来，虽然yan并没有对应的成绩记录，但是由于采用的是连接类型为左（外）连接，所以仍然把她放到了结果集中，只不过在对应的成绩记录的各列使用NULL值填充而已。

右（外）连接的语法

右（外）连接和左（外）连接的原理是一样的，语法也只是把LEFT换成RIGHT而已：

```
SELECT * FROM e1  
RIGHT [OUTER] JOIN e2 ON 连接条件 [WHERE 普通过滤条件];
```

只不过驱动表是右边的表e2，被驱动表是左边的表e1。

内连接的语法

内连接和外连接的根本区别就是在驱动表中的记录不符合ON子句中的连接条件时不会把该记录加入到最后的结果集，一种最简单的内连接语法，就是直接把需要连接的多个表都放到FROM子句后边。其实针对内连接，MySQL提供了好多不同的语法：

```
SELECT * FROM e1 [INNER | CROSS] JOIN e2 [ON 连接条件] [WHERE 普通过滤条件];
```

也就是说在MySQL中，下边这几种内连接的写法都是等价的：

```
SELECT * FROM e1 JOIN e2;

SELECT * FROM e1 INNER JOIN e2;

SELECT * FROM e1 CROSS JOIN e2;
```

上边的这些写法和直接把需要连接的表名放到FROM语句之后，用逗号分隔开的写法是等价的：

```
SELECT * FROM e1, e2;
```

再说一次，由于在内连接中ON子句和WHERE子句是等价的，所以内连接中不要求强制写明ON子句。

我们前边说过，连接的本质就是把各个连接表中的记录都取出来依次匹配的组合加入结果集并返回给用户。不论哪个表作为驱动表，两表连接产生的笛卡尔积肯定是一样的。而对于内连接来说，由于凡是不符合ON子句或WHERE子句中的条件的记录都会被过滤掉，其实也就相当于从两表连接的笛卡尔积中把不符合过滤条件的记录给踢出去，所以对于内连接来说，驱动表和被驱动表是可以互换的，并不会影响最后的查询结果。

但是对于外连接来说，由于驱动表中的记录即使在被驱动表中找不到符合ON子句条件的记录时也要将其加入到结果集，所以此时驱动表和被驱动表的关系就很重要了，也就是说左外连接和右外连接的驱动表和被驱动表不能轻易互换。

1.2.2.MySQL对连接的执行

复习了连接、内连接、外连接这些基本概念后，我们需要理解MySQL怎么样来进行表与表之间的连接，才能明白有的连接查询运行的快，有的却慢。

1.2.2.1.嵌套循环连接（Nested-LoopJoin）

我们前边说过，对于两表连接来说，驱动表只会被访问一遍，但被驱动表却要访问到好多遍，具体访问几遍取决于对驱动表执行单表查询后的结果集中的记录条数。

对于内连接来说，选取哪个表为驱动表都没关系，而外连接的驱动表是固定的，也就是说左（外）连接的驱动表就是左边的那个表，右（外）连接的驱动表就是右边的那个表。

如果有3个表进行连接的话，那么首先两表连接得到的结果集就像是新的驱动表，然后第三个表就成为了被驱动表，可以用伪代码表示一下这个过程就是这样：

```

for each row in e1 {    #此处表示遍历满足对e1单表查询结果集中的每一条记录，N条
    for each row in e2 {    #此处表示对于某条e1表的记录来说，遍历满足对e2单表查询结果集中的
        每一条记录，M条
            for each row in t3 {    #此处表示对于某条e1和e2表的记录组合来说，对t3表进行单
                表查询，L条
                    if row satisfies join conditions, send to client
            }
        }
    }
}

```

这个过程就像是一个嵌套的循环，所以这种驱动表只访问一次，但被驱动表却可能被多次访问，访问次数取决于对驱动表执行单表查询后的结果集中的记录条数的连接执行方式称之为**嵌套循环连接（Nested-Loop Join）**，这是最简单，也是最笨拙的一种连接查询算法，时间复杂度是 $O(N * M * L)$ 。

1.2.2.2.使用索引加快连接速度

我们知道在嵌套循环连接的步骤2中可能需要访问多次被驱动表，如果访问被驱动表的方式都是全表扫描的话，那速度肯定会很慢很慢。

但是查询e2表其实就相当于一次单表查询，我们可以利用索引来加快查询速度。回顾一下最开始介绍的e1表和e2表进行内连接的例子：

```
SELECT * FROM e1, e2 WHERE e1.m1 > 1 AND e1.m1 = e2.m2 AND e2.n2 < 'd';
```

我们使用的其实是嵌套循环连接算法执行的连接查询，再把上边那个查询执行过程表回顾一下：

查询驱动表e1后的结果集中有两条记录，嵌套循环连接算法需要对被驱动表查询2次：

当e1.m1 = 2时，去查询一遍e2表，对e2表的查询语句相当于：

```
SELECT * FROM e2 WHERE e2.m2 = 2 AND e2.n2 < 'd';
```

当e1.m1 = 3时，再去查询一遍e2表，此时对e2表的查询语句相当于：

```
SELECT * FROM e2 WHERE e2.m2 = 3 AND e2.n2 < 'd';
```

可以看到，原来的e1.m1 = e2.m2这个涉及两个表的过滤条件在针对e2表做查询时关于e1表的条件就已经确定了，所以我们只需要单单优化对e2表的查询了，上述两个对e2表的查询语句中利用到的列是m2和n2列，我们可以在e2表的m2列上建立索引。

新建 保存 另存为 添加索引 删除索引			
栏位	索引	外键	触发器
名	索引	索引类型	索引方法
idx_m2	m2	Normal	BTREE

因为对m2列的条件是等值查找，比如e2.m2= 2、e2.m2 = 3等，所以可能使用到**ref**的访问方法，假设使用ref的访问方法去执行对e2表的查询的话，需要回表之后再判断e2.n2 < d这个条件是否成立。

```
1 EXPLAIN SELECT * FROM e1, e2 WHERE e1.m1 > 1 AND e1.m1 = e2.m2 AND e2.n2 < 'd';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	e1	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	3	33.33	Using w
1	SIMPLE	e2	(Null)	ref	idx_m2	idx_m.5		denc1		33.33	Using w

在n2列上建立索引，涉及到的条件是e2.n2 < 'd'，可能用到range的访问方法，假设使用range的访问方法对e2表的查询的话，需要回表之后再判断在m2列上的条件是否成立。

名	索引	外键	触发器	选项	注释	SQL 预览
idx_m2						
idx_n2						

名	索引	索引类型	索引方法
idx_m2	m2	Normal	BTREE
idx_n2	n2	Normal	BTREE

假设m2和n2列上都存在索引的话，那么就需要从这两个里边儿挑一个代价更低的去执行对e2表的查询。当然，建立了索引不一定使用索引，只有在二级索引 + 回表的代价比全表扫描的代价更低时才会使用索引。

id	select_type	table	partitions	type	possible_keys	key	key_len	ref
1	SIMPLE	e1	(Null)	ALL	(Null)	(Null)	(Null)	(Null)
1	SIMPLE	e2	(Null)	ref	idx_m2,idx_n2	idx_m2	5	denc1

另外，有时候连接查询的查询列表和过滤条件中可能只涉及被驱动表的部分列，而这些列都是某个索引的一部分，这种情况下即使不能使用eq_ref、ref、ref_or_null或者range这些访问方法执行对被驱动表的查询的话，也可以使用索引扫描，也就是index(索引覆盖)的访问方法来查询被驱动表。

1.2.2.3.基于块的嵌套循环连接 (Block Nested-Loop Join)

扫描一个表的过程其实是先把这个表从磁盘上加载到内存中，然后从内存中比较匹配条件是否满足。

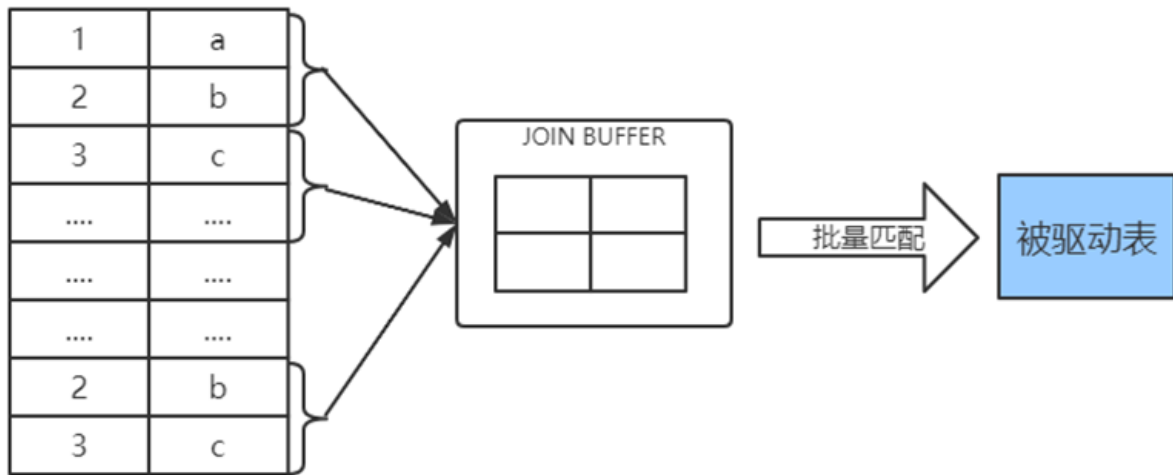
现实生活中的表成千上万条记录都是少的，几百万、几千万甚至几亿条记录的表到处都是。内存里可能并不能完全存放的下表中所有的记录，所以在扫描表前边记录的时候后边的记录可能还在磁盘上，等扫描到后边记录的时候可能内存不足，所以需要把前边的记录从内存中释放掉。

而采用嵌套循环连接算法的两表连接过程中，被驱动表可是要被访问好多次的，如果这个被驱动表中的数据特别多而且不能使用索引进行访问，那就相当于要从磁盘上读好几次这个表，这个I/O代价就非常大了，所以我们得想办法：尽量减少访问被驱动表的次数。

当被驱动表中的数据非常多时，每次访问被驱动表，被驱动表的记录会被加载到内存中，在内存中的每一条记录只会和驱动表结果集的一条记录做匹配，之后就会被从内存中清除掉。然后再从驱动表结果集中拿出另一条记录，再一次把被驱动表的记录加载到内存中一遍，周而复始，驱动表结果集中有多少条记录，就得把被驱动表从磁盘上加载到内存中多少次。

所以我们可不可以在把被驱动表的记录加载到内存的时候，一次性和多条驱动表中的记录做匹配，这样就可以大大减少重复从磁盘上加载被驱动表的代价了。

所以MySQL提出了一个join buffer的概念，join buffer就是执行连接查询前申请的一块固定大小的内存，先把若干条驱动表结果集中的记录装在这个join buffer中，然后开始扫描被驱动表，每一条被驱动表的记录一次性和join buffer中的多条驱动表记录做匹配，因为匹配的过程都是在内存中完成的，所以这样可以显著减少被驱动表的I/O代价。使用join buffer的过程如下图所示：



最最好的情况是join buffer足够大，能容纳驱动表结果集中的所有记录。

这种加入了join buffer的嵌套循环连接算法称之为**基于块的嵌套连接（Block Nested-Loop Join）**算法。

这个join buffer的大小是可以通过启动参数或者系统变量join_buffer_size进行配置，默认大小为262144字节（也就是256KB），最小可以设置为128字节。

```
show variables like 'join_buffer_size' ;
```

信息	结果1	概况	状态
Variable_name		Value	
▶ join_buffer_size		262144	

当然，对于优化被驱动表的查询来说，最好是为被驱动表加上效率高的索引，如果实在不能使用索引，并且自己的机器的内存也比较大可以尝试调大join_buffer_size的值来对连接查询进行优化。

另外需要注意的是，驱动表的记录并不是所有列都会被放到join buffer中，只有查询列表中的列和过滤条件中的列才会被放到join buffer中，所以再次提醒我们，最好不要把*作为查询列表，只需要把我们关心的列放到查询列表就好了，这样还可以在join buffer中放置更多的记录。